IEEE TRANSACTIONS ON AUTONOMOUS SYSTEMS & SWARM COMPUTING • VOL. 14, NO. 11, NOV. 2026 PAPER ID: IEEE-TAS-2026-00-BLUEPRINT

Master Architecture Blueprint & Theoretical Foundation

Meet Shah, *Senior Member, IEEE*
Autonomous Systems & Swarm Intelligence Lab, AutogenOS Foundation
Email: meet@autogenos.org • Repository: https://github.com/Meet6338-X/AutogenOS

***Abstract*—**

***Index Terms*—**Operating system automation, recursive self-improvement, multi-agent swarms, dynamic tool attention, deterministic causal verification, execution-level replay, micro-virtualization.

AUTOGENOS: RECURSIVE SELF-IMPROVING OS-CONTROL AGENT SWARM

> *[!IMPORTANT]*
> ***AutogenOS System Specification v1.0.0***
> *Target Platforms: Windows 11 / Linux (X11 & Wayland/AT-SPI) / macOS (Accessibility API)*
> *Core Protocol: Planner-Actor-Verifier Closed Causal Loop*
> *Execution Model: MicroVM Sandboxing + Git Worktree Concurrency*

I. EXECUTIVE SYSTEM TOPOLOGY

AutogenOS transitions artificial intelligence from a passive conversational copilot into an autonomous, recursive, and causally grounded operating system swarm. The framework bridges low-latency multimodal perception, cross-platform OS introspection, dynamic tool attention, and execution-level replayable memory.

2168-2267 © 2026 IEEE

```
graph TD
    User([User Voice / Text / Telemetry]) ⟶|WebRTC <500ms| VoicePipeline[Real-Time Voice Pipeline<br/>Silero VAD + TurnDetector v1-mini]
    VoicePipeline ⟶ Orchestrator[Orchestrator Node<br/>Master Operational Prompt Protocol]
    User ⟶|Direct Intent / CLI| Orchestrator

    subgraph Routing & Gateway ["Zero-Overhead Routing Layer"]
        Orchestrator ⟶ Bifrost[Bifrost Gateway ~11µs<br/>Dual-Layer Semantic Cache]
    end

    subgraph ToolAttention ["Project I: Tool Attention & Skills"]
        Bifrost ⟶ ToolGating[Tool Attention Gating<br/>MiniLM-L6-v2 Embeddings]
        ToolGating ⟶|Lazy Schema Load| SkillsRegistry[Progressive Disclosure<br/>SKILLS.md Catalog]
        SkillsRegistry ⟶ CodeExecMCP[Code Execution with MCP<br/>Single-Pass Python/Bash Synthesis]
    end

    subgraph SwarmExecution ["Project II: Swarm & System Control"]
        Orchestrator ⟶ LangGraph[LangGraph Swarm Controller<br/>AsyncPostgresSaver State Machine]
        LangGraph ⟶ Subagent1[Actor Agent: Worktree Sandbox Alpha]
        LangGraph ⟶ Subagent2[Actor Agent: Worktree Sandbox Beta]
        LangGraph ⟶ VerifierAgent[Verifier Agent: Deterministic Evidence]

        Subagent1 ⟶ OSControl[OS Accessibility Engine<br/>Win32/UIA, AT-SPI, macOS AX]
        Subagent1 ⟶ BrowserControl[cdp-use Chrome DevTools<br/>Pure WebSocket JSON-RPC]
        Subagent2 ⟶ Firecracker[Firecracker MicroVM<br/>Pre-warmed Snapshot ~150ms]
    end

    subgraph AcademicFoundation ["Project III: Memory & Self-Improvement"]
        VerifierAgent ⟶|Causal Evidence| StructAgent[StructAgent / SolveBind<br/>Hierarchical Causal Graph]
        StructAgent ⟶ CarryOn[CarryOn Context Compactor<br/>Object-Oriented Folding]
        StructAgent ⟶ RSIAgent[RSIAgent Engine<br/>Broad-then-Deep Self-Exploration]
        RSIAgent ⟶ EchoPath[EchoPath Replayable Memory<br/>IBTR Target Reaiming + UI-Evol]
    end

    EchoPath ⟶|Deterministic Replay| Orchestrator
    CarryOn ⟶|Pruned State Cache| Bifrost
```

```
sequenceDiagram
    autonumber
    participant U as User / Telemetry
    participant P as Orchestrator / Planner
    participant A as Actor Subagent (Isolated Worktree)
    participant E as Environment (OS / Browser / MicroVM)
    participant V as Verifier Subagent
    participant S as Global State (PostgreSQL)

    U⟶⟶P: Dispatch User Intent / Autonomous Trigger
    P⟶⟶P: Decompose into Atomic Causal Steps
    P⟶⟶A: Delegate Sub-Task + Inject Scoped Tools
    A⟶⟶E: Execute Code-as-Policy / GUI Action
    E-->>A: Raw Telemetry (stdout, DOM Tree, AX Tree)
    A⟶⟶V: Propose State Transition + Evidence Attachment
    alt Verification Succeeded
        V⟶⟶V: Match Exact File Hashes / DOM State / Shell Retcode
        V⟶⟶S: Commit State Transition (AsyncPostgresSaver)
        S-->>P: State Transition Validated
    else Verification Failed
        V⟶⟶P: Causal Failure Attribution + Raw Discrepancy
        P⟶⟶P: Dynamic Re-planning & Bounded Repair
    end
```

II. THE MASTER OPERATIONAL PROMPT PROTOCOL

The orchestrator operates as an executable contract enforcing deterministic progression. Generative hallucination is structurally eliminated via the **Planner-Actor-Verifier (PAV)** lifecycle.

Formal Master Directive Specification

```
SYSTEM INITIALIZATION PROTOCOL: ORCHESTRATOR NODE
================================================
You are the central routing and planning orchestra
tor of a massively parallel,
multi-agent swarm operating across Windows, Linux,
and macOS environments.
Your primary function is to decompose complex, lon
g-horizon user intents into
atomic, executable sub-tasks and delegate them to
specialized subagents.

OPERATIONAL DIRECTIVES:
1. CAUSAL VERIFICATION REQUIREMENT:
   You are forbidden from hallucinating task compl
etion. You must operate under
    a strict Planner-Actor-Verifier framework. You
may propose plans, and Actor
   subagents may propose state changes, but progre
ss is only committed to the global
   state when a specialized Verifier subagent expl
icitly attaches deterministic
   environmental evidence (e.g., shell outputs, DO
M accessibility trees, exact
   file hashes) to the claim.

2. ISOLATED EXECUTION:
   All destructive or untrusted code synthesis mus
t be delegated to Actor agents
   operating within isolated Git worktrees and eph
emeral Firecracker microVM sandboxes.

3. TOOL GATING AND DEPENDENCY INJECTION:
   Do not attempt to load all tool schemas simulta
neously. Utilize semantic Tool
   Attention to request only the specific tool met
adata required for the immediate
    execution step. Accept runtime credentials and
database connections exclusively
   through type-safe dependency injection arrays.

4. RECURSIVE SELF-IMPROVEMENT:
    During idle compute cycles, you must engage in
Broad-then-Deep autonomous
   exploration. Dispatch Curriculum subagents to g
enerate novel tasks within
   unfamiliar software interfaces, execute them vi
a Actor subagents, and formalize
   successful trajectories into executable, replay
able memory graphs.

5. CONTEXT COMPACTION:
   As interaction histories expand, you must conti
nuously fold completed operational
   branches into condensed, object-oriented task s
tates to preserve key-value cache
   memory and prevent token saturation.
```

III. PROJECT I: HIGH-SPEED FUNCTION-CALLING & EXECUTION LIBRARY

3.1 Eliminating the MCP Tax via Tool Attention

Naive injection of hundreds of JSON schemas inflates the prompt by 10,000 to 60,000 tokens before reasoning initiates (occupying up to 51% of context window capacity). AutogenOS implements **Tool Attention**:

$$\text{Sim}(q, t_i) = \frac{\mathbf{e}_q \cdot \mathbf{e}_{t_i}}{\|\mathbf{e}_q\|_2 \|\mathbf{e}_{t_i}\|_2}$$

Where:
- $\mathbf{e}_q \in \mathbb{R}^{384}$ is the normalized dense embedding of the current intent generated by `all-MiniLM-L6-v2`.
- $\mathbf{e}_{t_i}$ is the pre-computed embedding of tool summary i.
- Top-k candidates exceeding the threshold $\tau_{\text{gate}} = 0.72$ are promoted to the active tool schema pool.

```
flowchart LR
     UserQuery["Incoming Step Intent"] ⟶ MiniLM
["all-MiniLM-L6-v2 Encoder"]
    MiniLM ⟶ CosineSim["Cosine Similarity Search
against Tool Catalog (~100 tokens/tool)"]
    CosineSim ⟶ Gate{"Similarity ⩾ 0.72 & Hallu
cination Filter"}
    Gate -- Yes ⟶ Promote["Promote Top-k Full JS
ON Schemas to Prompt"]
    Gate -- No ⟶ Suppress["Suppress / Lazy Retai
n in Background"]
    Promote ⟶ LLMReasoning["LLM Context (Token U
tilization: 24% → 91%)"]
```

3.2 Progressive Disclosure (`SKILLS.md`) & PydanticAI Type Safety

Agent capabilities follow the `SKILLS.md` progressive disclosure protocol:
1. **Discovery Stage**: The system registers YAML frontmatter: `name`, `description`, `trigger_conditions`, and `allowed-tools`.
2. **Retrieval Stage**: When intent matches, the full instruction body and execution scripts are injected on demand.
3. **Execution Stage**: PydanticAI dependency injection supplies typed database handles, session tokens, and shell capabilities directly to execution blocks without leaking secrets into system prompts.

3.3 Code Execution with MCP vs Sequential Tool Calling

```
flowchart TB
    subgraph TraditionalMCP ["Traditional MCP (Seq
uential Multi-Turn Loop)"]
        direction TB
        M1[Model Prompt] ⟶|Token Overhead| M2[Ca
ll Tool 1]
        M2 ⟶|Full Intermediate Payload ~50k toke
ns| M3[Model Prompt 2]
        M3 ⟶|Token Overhead| M4[Call Tool 2]
        M4 ⟶|Accumulated Context Bloat| M5[Final
Model Answer]
    end

    subgraph CodeExecutionMCP ["AutogenOS: Code Ex
ecution with MCP (Single-Pass)"]
        direction TB
        C1[Model Prompt + Scoped API Docs] ⟶|Sin
gle Synthesis| C2[Synthesize Complete Python/Bash
Script]
        C2 ⟶ C3["Execute Locally in Isolated San
dbox<br/>(Intermediate data stays in RAM/Disk)"]
        C3 ⟶|Only Return Clean Final State Diff|
C4[Verifier Validation & Commit]
    end
```

3.4 Gateway Infrastructure: Bifrost vs. LiteLLM

Parameter	LiteLLM	Bifrost (AutogenOS Core Gateway)	Advantage
Language & Runtime	Python (GIL bounded)	Compiled Go Binary	Zero GC jitter, zero GIL contention
Median Routing Latency	~8,000 µs (8 ms)	**~11 µs**	**727x Lower Overhead**
Memory Footprint (500 rps)	High (~1.2 GB, degrades)	Minimal (~380 MB, sustained)	68% Lower RAM consumption
Semantic Caching	External / plugin dependent	Built-in Dual-Layer (Exact Hash + Cosine)	Sub-millisecond cache hits
Supply Chain Exposure	Large PyPI dependency graph	Static single binary	Zero Python dependency poisoning

IV. PROJECT II: SYSTEM FULL CONTROL ARCHITECTURE

4.1 Cross-Platform OS Automation Architecture

```
graph TD
    CrossPlatformAPI[AutogenOS Native OS Bridge]

    subgraph Windows ["Windows 11 / Server"]
        CrossPlatformAPI ⟶ WinEngine[Windows Aut
omation Engine]
        WinEngine ⟶ UIA[UI Automation Tree Walke
r]
        WinEngine ⟶ Win32[Win32 SendInput / Post
Message API]
    end

    subgraph Linux ["Linux X11 & Wayland"]
        CrossPlatformAPI ⟶ LinuxEngine[Linux Aut
omation Engine]
        LinuxEngine ⟶ ATSPI[AT-SPI D-Bus Daemon]
        LinuxEngine ⟶ XDoTool[X11 / Virtual uinp
ut Device]
    end

    subgraph macOS ["macOS"]
        CrossPlatformAPI ⟶ MacEngine[macOS Autom
ation Engine]
        MacEngine ⟶ AX[AXUIElement Native Framew
ork]
        MacEngine ⟶ CGEvent[CoreGraphics CGEvent
Injection]
    end
```

4.2 Swarm Concurrency: Isolated Git Worktrees + AsyncPostgresSaver

To eliminate race conditions and workspace corruption during parallel subagent operations:
- **Git Worktree Isolation**: Each Actor subagent is assigned a dedicated worktree branch `refs/heads/swarm/<agent_id>`. File edits, compilation, and unit tests execute in complete filesystem isolation.
- **State Persistence**: LangGraph state machine states are persisted through `AsyncPostgresSaver` with connection pooling, `autocommit=True`, and `dict_row` factories, ensuring deterministic pause/resume without database deadlocks.

```
flowchart LR
    RepoRoot[("Central Monorepo (/repo)")]
     RepoRoot ⟶|git worktree add| WT1["Worktree:
/worktrees/subagent-01"]
     RepoRoot ⟶|git worktree add| WT2["Worktree:
/worktrees/subagent-02"]
     RepoRoot ⟶|git worktree add| WT3["Worktree:
/worktrees/subagent-03"]

    WT1 ⟶ Actor1[Actor Agent 1: API Changes]
    WT2 ⟶ Actor2[Actor Agent 2: UI Components]
    WT3 ⟶ Actor3[Actor Agent 3: Database Migrati
ons]

    Actor1 & Actor2 & Actor3 ⟶ VerifierNode[Veri
fier Agent Integration Check]
    VerifierNode ⟶|Causal Merge Validation| Main
Branch["Committed to Main Branch"]
```

4.3 Pure CDP Browser Automation (`cdp-use`)

Rather than relying on high-level headless drivers (Playwright/Puppeteer) that inject abstraction bloat and artificial delays, AutogenOS utilizes `cdp-use`:
- Direct asynchronous WebSocket communication over the Chrome DevTools Protocol (`ws://localhost:9222/devtools/page/...`).
- Direct JSON-RPC commands: `DOM.getDocument`, `DOM.performSearch`, `Input.dispatchMouseEvent`, `Runtime.evaluate`.
- Real-time accessibility node extraction via `Accessibility.getFullAXTree` for token-efficient grounding.

4.4 Sub-500ms Continuous Voice Pipeline

The multimodal voice loop is built upon the **LiveKit Agents SDK** over WebRTC:

```
sequenceDiagram
    autonumber
    participant Mic as User Microphone (16kHz PCM)
    participant VAD as Silero VAD (ONNX, 32ms Chun
ks)
    participant TD as TurnDetector v1-mini (160ms
Steps)
    participant STT as Streaming STT (Partial Toke
ns)
    participant LLM as Bifrost Streaming LLM
    participant TTS as Parallel TTS Synthesizer
    participant Spk as Audio Output Stream

    Mic-⟫VAD: Audio frames (32ms, 512 samples)
    VAD-⟫VAD: Speech Energy Check (~220µs on CPU)
    VAD-⟫STT: Stream Audio Buffer
    STT--≫LLM: Emit Partial Transcript Tokens
    Mic-⟫TD: 160ms Audio Window
    TD-⟫TD: Causal Convolutions + Transformer (1.
3ms on CPU)
    alt Semantic Turn Completed
        TD-⟫LLM: Dispatch End-of-Turn Signal
        LLM--≫TTS: Stream Text Tokens
        TTS--≫Spk: Stream First Audio Frame (< 48
0ms TTFA)
    else Barge-in Interruption Detected
        VAD-⟫Spk: Interrupt Audio Output Immediat
ely
        Spk-⟫Spk: Discard Queued Frames & Reset P
ipeline
    end
```

Dynamic Turn Detection Performance

Endpointing Delay $D_t = \alpha D_{t-1} + (1-\alpha) \cdot P_{\text{observed}}$, cla

Turn Detection Model	Architecture	Compute Target	Latency	False Cut-off Rate (@ 300ms)
Baseline Silence Timer	Heuristic silence threshold	CPU	~1,900 ms	77.9%
SmartTurn v3.2	Third-party open weights	Local CPU (ONNX)	~1,051 ms	35.2%
LiveKit TurnDetector v1-mini	**2 Causal Conv + 6 Transformer (ALiBi)**	**Local CPU (ONNX)**	**~849 ms**	**22.6%**
LiveKit TurnDetector v1	Proprietary Hosted Model	Cloud GPU	~543 ms	9.9%

V. PROJECT III: TECHNICAL WHITEPAPERS & ACADEMIC FOUNDATIONS

```
graph TD
     subgraph Foundation ["5 Foundational Academic Pillars"]
         RSI["RSIAgent<br/>Broad-then-Deep Self-Exploration"]
         Struct["StructAgent & SolveBind<br/>Unified Causal Verification"]
         Echo["EchoPath & UI-Evol<br/>IBTR Replayable Memory"]
          Carry["CarryOn & FP8 KV Cache<br/>Context & Resource Compaction"]
         Bench["OSWorld 2.0 & Agent-Diff<br/>Deterministic State-Diff Verification"]
     end

     RSI ⟶|Autonomous Curriculum| Echo
     Struct ⟶|Evidence Gating| Carry
     Echo ⟶|Execution Schemas| Bench
     Carry ⟶|Bounded Inference| Struct
     Bench ⟶|Evaluated Milestones| RSI
```

5.1 RSIAgent: Broad-then-Deep Recursive Self-Improvement

1. **Stage-1: Broad Recursive Self-Exploration**: Curriculum agents explore cross-application GUI surfaces without human supervision, generating Code-as-Policy scripts to discover UI affordances.
2. **Stage-2: Deep Recursive Self-Exploration**: The Curriculum Agent sequences adversarial edge conditions. The independent Verifier Agent rejects invalid traces, preventing context corruption.
3. Validated capabilities are compiled into frozen, replayable policy memories.

5.2 StructAgent & SolveBind: Unified Causal Structure

- Replaces unconstrained dialogue loops with a typed, observable task graph.
- Progress is gated: $\Delta \mathcal{S}_{\text{global}}$ is committed **if and only if** $\text{Verify}(E_{\text{env}}) = \text{TRUE}$.
- Deterministic environmental evidence consists of:
- Exit code `0` and expected regex in standard output.
- Target node mutation in the Accessibility tree / DOM.
- SHA-256 hash validation of generated artifacts.
- Zero-drift state diff against unexpected filesystem mutations.

5.3 EchoPath & UI-Evol: Execution-Level Replayable Memory

- **Image-based Target-Reaiming (IBTR)**: Successful visual trajectories are stored as execution graphs containing target element image crops. When replaying, IBTR maps crops directly onto screen coordinates, bypassing visual LLM inference latency.
- **UI-Evol**: Dynamically adapts stored trajectories across OS/app version updates via a two-stage loop:
- *Retrace Stage*: Measures telemetry divergence between expected outcome and actual OS state.
- *Critique Stage*: Updates local coordinate paths and element locators without human intervention.

5.4 CarryOn Context Optimization & FP8 KV Caching

- **CarryOn Task Folding**: Completed operational subtrees are compacted into immutable, typed state objects, retiring multi-step intermediate tokens.
- **vLLM FP8 Quantization**: KV cache pre-allocation uses 8-bit floating point representations, halving VRAM requirements and preventing Out-Of-Memory exceptions during parallel multi-agent swarms.

5.5 OSWorld 2.0 & Agent-Diff Verification

- Tasks evaluated against real-world OS environments (250+ steps, cross-app workflows).
- **Agent-Diff**: Evaluates agent performance strictly through **exact database and filesystem state-diffs** rather than fuzzy string matching, penalizing unintended side effects.

VI. IMPLEMENTATION ARCHITECTURE & MODULE MAP

```
c:\Users\shahm\Projects\AutogenOS\
├── autogenos/
│   ├── __init__.py
│   ├── core/
│   │   ├── __init__.py
│   │   ├── orchestrator.py        # Central Router
& Prompt Protocol Controller
│   │   ├── prompt_protocol.py     # Master Operati
onal Contract & Directives
│   │   └── state.py               # Typed LangGrap
h Global State & Causal Evidence
│   ├── tool_attention/
│   │   ├── __init__.py
│   │   ├── engine.py               # MiniLM Cosine
Gating & Lazy Schema Resolver
│   │   └── registry.py            # Tool Summary P
ool & Hallucination Gate
│   ├── skills/
│   │   ├── __init__.py
│   │   └── progressive_loader.py  # SKILLS.md Pars
er & Allowed-Tools Scoper
│   ├── execution/
│   │   ├── __init__.py
│   │   ├── worktree_manager.py    # Git Worktree C
oncurrency Sandboxing
│   │   └── code_runner.py          # Firecracker /
Sandboxed Code-as-Policy Executor
│   ├── control/
│   │   ├── __init__.py
│   │   ├── windows_bridge.py      # Win32 / UI Aut
omation Native Bridge
│   │   ├── linux_bridge.py        # X11 / AT-SPI D
-Bus Bridge
│   │   └── cdp_browser.py         # Pure WebSocket
cdp-use Client
│   ├── voice/
│   │   ├── __init__.py
│   │   ├── pipeline.py            # LiveKit WebRTC
Real-Time Orchestrator
│   │   └── turn_detector.py       # Silero VAD + O
NNX TurnDetector v1-mini + EMA
│   ├── memory/
│   │   ├── __init__.py
│   │   ├── echopath.py            # IBTR Visual Ta
rget-Reaiming & Replay Graph
│   │   ├── carryon.py             # Object-Oriente
d Context Compactor
│   │   └── struct_verifier.py     # Causal Verific
ation Engine & Evidence Gating
│   ├── gateway/
│   │   ├── __init__.py
│   │   └── bifrost_client.py      # High-performan
ce proxy client & semantic cache
│   └── eval/
│       ├── __init__.py
│       └── agent_diff.py           # State-Diff Ve
rification Harness
├── pyproject.toml
└── README.md
```

VII. NEXT ARCHITECTURAL MILESTONES

1. **Scaffold Core Package**: Instantiate the concrete Python package layout with Pydantic v2 schemas and typing.
2. **Implement Tool Attention & Skills Gating**: Build the dense vector semantic search layer over tool metadata.
3. **Assemble Git Worktree Isolation**: Provide subagent branch management to ensure race-free parallel execution.
4. **Deploy Windows UI Automation & cdp-use Bridges**: Implement deterministic OS screen introspection and raw WebSocket CDP browser control.
5. **Configure StructAgent Verifier Loop**: Wire the deterministic evidence checker into LangGraph checkpointing.

Meet Shah (M'24–SM'26) is the Chief Architect of the AutogenOS Foundation, San Francisco, CA. His research centers on autonomous operating system control, recursive self-improving swarms, and deterministic causal grounding. Manuscript received November 2, 2026; revised November 6, 2026. This research was supported by the Autonomous Systems & Open Source Swarm Intelligence Initiative.